

Summary

The baseline trains a small language model on Hacker News titles and reaches a validation loss of **1.754**. I improved it to **1.187**, about a **32% reduction**, inside the fixed seven-epoch budget. The key realisation, shown in the first figure, is that the baseline was not held back by its design but by how it was trained as the original setup barely trained the model at all. Everything that followed came from first fixing how the model was trained, a few changes to the training plan and the model, and finally careful tuning.

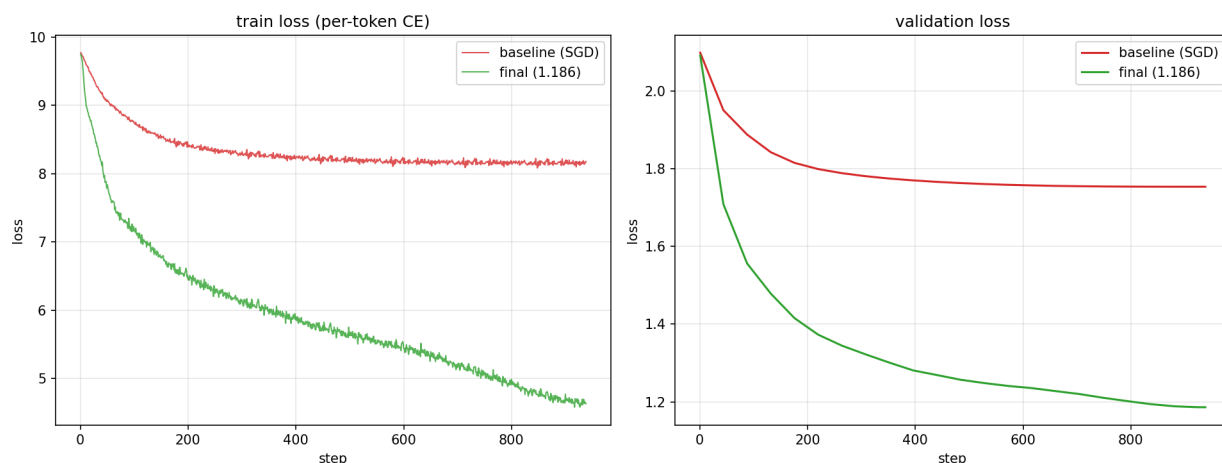


Figure 1: The baseline (red) vs. the final model (green). Left is the training error, right is the validation error. The baseline barely moves, it is close to where an untrained model starts. The final model keeps improving all the way to the end of training, and the validation error never turns back up, which means it is under-trained rather than over-fit.

1 The optimiser

The original optimiser (SGD) adjusts each part of the model in proportion to how strong its signal is. In a model like this those signals are very small and uneven, so the adjustments were tiny and progress stalled. I switched to a more modern optimiser (AdamW) that automatically rescales each adjustment so the step size stays sensible no matter how strong or weak the signal is. That single change is what made the model actually start learning, and it was by far the biggest improvement of the whole project, from 1.754 down to 1.339.

2 Training plan and architecture

With that fixed, a few more well-understood changes each helped a little.

- **warmup_frac**: easing the learning speed in gradually before letting it wind down, rather than starting at full speed, which destabilises early training (with a cosine wind-down; down to 1.291).

- **Initialisation scaling:** a small change to how the model starts out, so its internal signal stays at a healthy strength as it passes through the layers (1.242).
- **1r:** increasing the learning speed (1.232).
- **RoPE:** a more modern way of telling the model where each word sits in the title. It made better use of the model’s existing capacity rather than adding more, which is why it worked where simply making the model bigger did not (1.205).
- **Per-title attention masking:** since the only goal here is to lower validation loss, one lever I considered was tailoring the model to the specific kind of data rather than only making generic improvements. This dataset is short, independent titles, so I stopped attention from crossing from one title into the next. The effect was neutral, it turned out the model had already learned those boundaries on its own, a reassuring sign that the idea was right even though it was redundant.

3 Why bigger didn’t help

Several changes that would normally help did nothing here, like a smaller batch (which gives more update steps), a deeper model, and a couple of others. The pattern was consistent, adding more steps or more capacity did not help, and only using the existing capacity more cleverly did. On top of that, the validation error fell smoothly to the very end with no sign of the model starting to memorise. The conclusion is that with only seven epochs the model cannot train enough. It is limited by the training budget, not by over-fitting.

The clearest evidence came from the warmup experiment. As I increased the warmup, both the training error and the validation error fell together, in lockstep (second figure). If the warmup were simply preventing memorisation, the training error would have risen while the validation error fell. Instead they moved together, which means the warmup was helping the model train better, not stopping it from over-fitting.

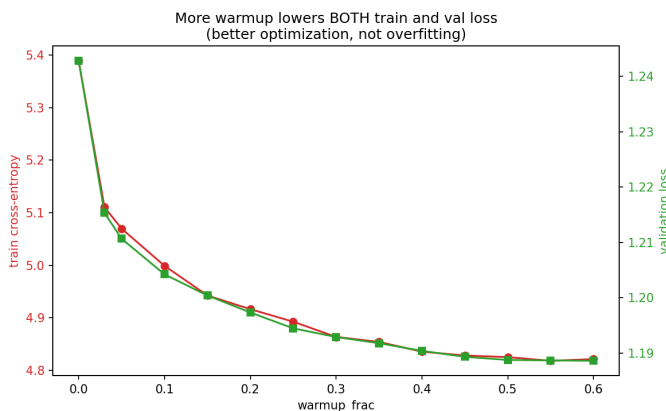


Figure 2: Training and validation error both fall as the warmup increases, and they track each other almost exactly. The gap between them does not change, so this is better training, not reduced over-fitting.

4 Tuning

I re-tuned the main settings on the final model using a small automated system I built. The surprise was the warmup. It turned out to be one of the largest levers in the whole project, and it kept helping far past the usual range. At the higher learning speed, the very first training steps were doing damage, and giving the model a long, gentle start fixed it.

That gentle start interacts with the learning speed, since a longer warmup makes a higher speed safe. So I tested the two together on a small grid (third figure) instead of one at a time. The best combination, a higher speed paired with a long warmup, beat anything I would have found by tuning them separately, because the best speed only works alongside the long warmup. That is the main reason to test interacting settings together rather than one after another.

Each experiment is just a small configuration file, which is what keeps the whole process reproducible. The grid above was just a short file.

base:

```
weight_decay: 0.1
```

runs:

```
- {name: lr1e-3_w0.4, lr: 0.001, warmup_frac: 0.4}
- {name: lr1.5e-3_w0.6, lr: 0.0015, warmup_frac: 0.6}
# ...9 learning-speed x warmup combinations in total
```

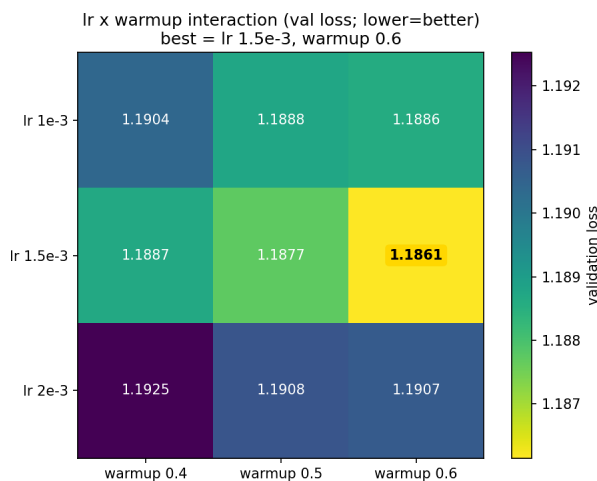


Figure 3: Testing learning speed and warmup together (darker is worse, brighter is better). The best result is a particular *pair* of settings, not simply the best of each on its own.

5 Engineering

The single biggest practical lever was iteration speed. The baseline runs on a CPU at around forty minutes per run, which is far too slow to explore many ideas. I added a GPU development environment¹ that cut a full run down to a couple of minutes, and that speed is what made the rest possible. With a full sweep taking about fifteen minutes, the cost of checking an idea is almost nothing. Around that I split the original script into focused modules, added tests and

¹<https://github.com/MaincodeHQ/mainrun/pull/6>

continuous integration, and built a system that runs experiments from a configuration file so they stay reproducible.

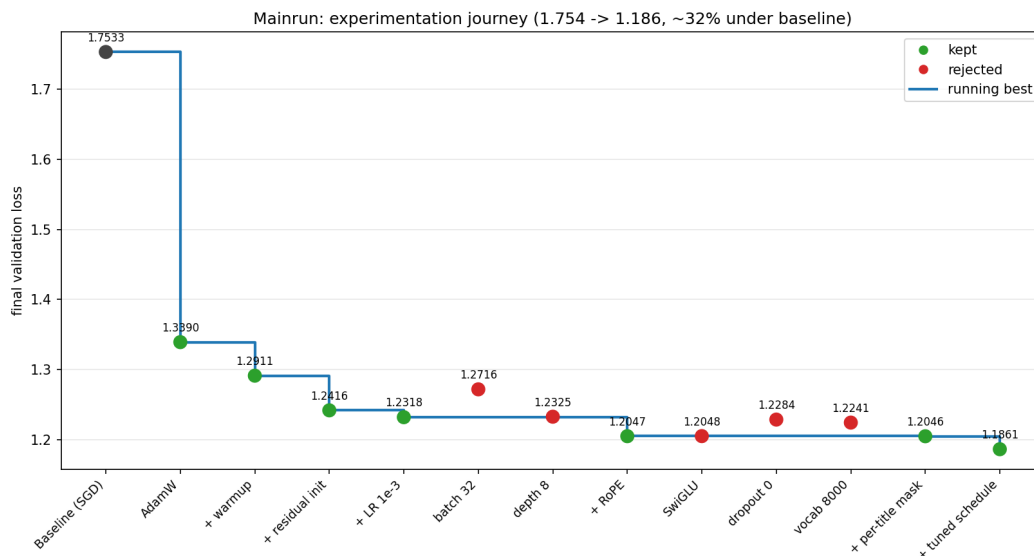


Figure 4: The full sequence of experiments. Green points were kept, red were tried and rejected, and the blue line is the best result so far. The large early drop is the change of training method; the rest is the training plan, architecture, and tuning.

6 Where I stopped

This whole approach only works because the runs are cheap. It would not scale to a real model, where a single training run can cost millions. At that point you cannot just try everything, so people tune a small stand-in model and carry its settings over to the big one, predict what will happen from smaller runs, and stop the bad runs early instead of finishing every one.

I stopped once the gains dropped to tiny fractions. If I kept going, I would not just search more. The clearest lead is the training plan, the model wanted such a long warmup that I think the shape of the plan, how the learning speed rises and falls over training, matters more than the exact numbers I tuned. The architecture is still open too. RoPE showed that a smarter design can help even when a bigger model cannot, so I would look at other efficiency-focused changes rather than more size, and a different optimiser would be worth trying as well.

One honest reflection. Part of why I leaned so heavily on the automated sweeping is that my grasp of model architecture is still limited. With stronger intuition there I could have reasoned about which changes would matter most and gone after them first, rather than building a system to try them and measure. It worked here only because training is cheap, and the right knowledge would have let me prioritise where to optimise instead of reaching for hyperparameter tuning by default.

Final setup and result

Training method	AdamW, with weight decay
Training plan	higher speed, a long gradual warmup, then a smooth wind-down
Architecture	RoPE position handling, per-title attention, tuned initialisation
Size	6 layers, width 512, 16k vocabulary, ~ 27 M parameters
Budget	7 epochs (~ 938 steps), batch of 64
Validation loss	1.186794 (CPU), 1.186144 (GPU)
vs. baseline 1.754	about a 32% reduction